

Barline Detection & Measure Numbering – Models Comparison

Goal: Identify OMR models and architectures that excel at **detecting barlines and inferring measure structure** in scanned sheet music, improving on the current Oemer/HOMR pipeline.

Top 5 Promising Models (Comparison Table)

Model / Approach	Methodology	Barline Detection (Accuracy & FP)	Measure Structure Support	License & Feasibility
Oemer/HOMR (UNet pipeline)	Two-stage CNN segmentation + heuristics 1 2	Good recall; uses staff context to filter stems, but some false barlines remain 3 . Relies on post-processing rules.	Outputs MusicXML with measures; measure numbering automatic, but errors if FP barlines 4 .	Oemer MIT / HOMR AGPL. Runs on RTX 4060 (ONNX/TF). Pre-trained on CVC-MUSCIMA & DeepScores 5 . Moderate training effort to fine-tune.
YOLO-based Symbol Detector	Object detection (e.g. YOLOv8) on DeepScores 6 7	High precision for clear barlines; stems as separate class still challenging 8 . 9 . Can tune confidence to reduce FP.	Detects barline objects explicitly. Can also train to detect whole measure regions 10 . Measure numbering by counting detected barlines in order.	Ultralytics YOLOv8 (GPL) or reimplement via Detectron2 (Apache). Feasible on RTX 4060. Pre-trained weights available; fine-tune on ~few thousand images 11 .

Model / Approach	Methodology	Barline Detection (Accuracy & FP)	Measure Structure Support	License & Feasibility
Mask R-CNN Instance Segmentation	Instance segmentation (Mask R-CNN) on OMR datasets ¹² ¹³	Precise delineation of barline pixels, even when overlapping symbols ¹⁴ . Reduces confusion with stems by separate masks. Achieved ~60% mAP on dense scores ¹⁵ ¹⁶ .	Detects individual barline segments as objects. No built-in measure grouping, but coordinates can be grouped into measures. Facilitates measure labeling once barlines are identified.	Matterport/ Detectron2 (Apache) implementations available. Pre-trained on COCO; domain pre-training on MUSCIMA++ boosts accuracy ¹³ . Training on ~1k+ images yielded ~59.7% mAP ¹⁵ .
DETR + Notation Assembly	Transformer-based DETR detection with relationship prediction ¹⁷ ¹⁸	Robust: Learns global context – stems attach to noteheads, barlines stand alone. Greatly filters stem-vs-barline FPs by relational reasoning. (E.g. vertical line with no notehead relation → barline.)	Implicit measure structure via detected “measure separator” class ¹⁹ and staff grouping. Can output measure boundaries if annotated (MUSCIMA++ provides measureSeparator labels). Measure numbering derived from sequence of barline objects per staff.	DETR (Apache). Research prototype in Python ¹⁷ . Requires moderate GPU (ResNet-50 backbone). Needs training data with symbol relations (e.g. MUSCIMA++ graph) – high effort, but one model does detection + assembly.

Model / Approach	Methodology	Barline Detection (Accuracy & FP)	Measure Structure Support	License & Feasibility
End-to-End OMR (Seq-to-Seq)	Full-page sequence transcription via CNN + Transformer/CRNN ²⁰ ²¹	Infers barlines as part of transcription. Few false barlines – model only inserts barlines where musically plausible. Might miss faint barlines if not learned. Overall accuracy tied to sequence fidelity.	Predicts complete measures in order. Measure numbering implicit – transcribed output contains barlines (e.g. “	” or MusicXML measure tags) in sequence ²² ²¹. Ensures rhythmic consistency across measures.

Table: Comparison of leading approaches for barline detection and measure segmentation in OMR.

1. Oemer/HOMR – UNet Segmentation Pipeline

Overview: Oemer and its successor HOMR use a multi-stage CNN pipeline. The first UNet separates staff lines from symbols; the second UNet segments symbols into classes (noteheads, rests, clefs, **stems** etc) ¹. Heuristic algorithms then extract barlines by combining the two outputs ²³. HOMR builds on Oemer with a stronger model and a Transformer-based notation stage for robust output ² ²⁴.

Barline Detection: Oemer/HOMR explicitly tackle the stem vs barline confusion by leveraging staff line info and object length. The pipeline recognizes that true barlines span the full staff height, whereas note stems are shorter. In Oemer, the second UNet often mislabels barlines as short “stems” due to dataset quirks ³. The fix is to use the first UNet’s staff map to check vertical lines’ extents and filter out those that don’t cross all staff lines ⁴. This reduces false positives significantly. HOMR’s documentation confirms it **identifies clefs and bar lines as anchor symbols** for assembling staves ²⁴, indicating barline detection is integral. In practice, Oemer/HOMR achieve decent barline recall, but some false barlines persist (e.g. tall noise blobs or connected stems that pass the length filter).

Measure Numbering: The pipeline outputs MusicXML with measures. Once barlines are detected per staff, Oemer groups them across staves (systems) and numbers measures sequentially ²⁵. However, any spurious barline will cause an extra (empty) measure in the output. For example, a misclassified symbol can lead to an incorrect measure split ²⁶. HOMR’s use of a Transformer for the final recognition might implicitly validate rhythms, but it still relies on the initial barline segmentation. Overall, measure numbering is **straightforward** (counting intervals between barlines) but only as accurate as the barline detection.

Strengths: End-to-end pipeline; no manual intervention needed. The dual-UNet approach is robust to skew and noise (with built-in deskewing and augmentation) ²⁷ ²⁸. It already runs in your environment

(3–5 minutes/page on GPU) ²⁹. License-wise, Oemer is MIT (commercial-friendly), and although HOMR is AGPL ³⁰, you could use HOMR for research and still integrate ideas into Oemer to avoid GPL restrictions.

Weaknesses: The heuristic post-processing is at its limit – as you noted, further FP reduction is unsafe without risking true barlines. The “dataset bug” in training (barlines learned as short segments) indicates the model wasn’t explicitly trained to classify full barlines ³. Thus, it relies on rules that sometimes fail for borderline cases (e.g. a slightly broken barline might be discarded as two short segments, or a thick stem might be mistaken as a thin barline). Also, being a multi-stage system, errors can propagate: if staff line detection fails or a barline is broken by staff removal, the later stages may miss it entirely. Finally, while it outputs measures, it doesn’t validate them – it won’t catch a missing barline due to rhythmic inconsistency.

Applicability: Oemer/HOMR form a solid baseline. They could be improved by retraining or fine-tuning the segmentation with better barline labels. For instance, adding a dedicated “barline” channel in the second UNet (rather than lumping with stems) could directly reduce confusion. This would require assembling a training set where barlines are labeled distinctly (MUSCIMA++ or DeepScores could provide such annotations). Given the existing code (Oemer has a training script) ³¹, you could attempt to fine-tune its model on a small custom dataset of printed scores with ground-truth barline masks. In summary, **HOMR/Oemer is a strong foundation**, but to transcend its heuristic limits, more advanced models (below) offer a path forward.

2. YOLO-Based Symbol Detection (DeepScores)

Overview: Object detection models like YOLO treat musical symbols (including barlines) as objects to localize with bounding boxes. There is active research applying YOLO and Faster R-CNN to OMR ³² ⁶. For example, recent work trained YOLOv8 on the **DeepScores** dataset (a large set of synthetic printed music annotations) ⁶. They trained one YOLO model to detect all symbols, and even a second YOLO to detect entire measure regions ⁷ ¹⁰. These detectors operate on full page images (often resized to ~2000px on the long side) and output classes like “barline”, “notehead”, etc., with confidence scores.

Barline Detection: YOLO can achieve high precision on barlines, as these are relatively distinctive vertical lines. In the DeepScores-trained model, **barlines were a labeled class**, separate from stems. The comparative study by Pacha et al. showed one-stage detectors can robustly detect symbols but might struggle with very thin or numerous objects ³³ ³⁴. In practice, YOLOv8 was able to reach an overall mAP~72.8% (IOU=0.5) across many symbol classes ³⁵ ³⁶. Barlines specifically were not listed among the failure cases – the worst classes were stems and ledger lines (which had near 0% recall due to their abundance and overlap) ⁸ ⁹. This indicates YOLO can distinguish barlines from stems if trained properly: stems gave it trouble (likely because they overlap with noteheads and were extremely frequent), whereas barlines, being longer and less frequent, likely achieved decent precision. Indeed, **barlines did not appear in the “worst 10 classes” list** ⁸ ⁹, suggesting the model detected most barlines with some success. A well-tuned YOLO can use non-maximum suppression to eliminate duplicate detections and a confidence threshold to drop spurious predictions. For example, any detection that looks like a vertical line but has low confidence can be filtered, thereby addressing false positives to an extent. Moreover, because YOLO’s convolutional backend sees context, it can learn that a vertical line through a staff with noteheads attached is likely a stem (not a barline). This contextual learning is a big improvement over pure heuristic length filtering.

Measure Numbering: A detector alone gives barline positions; measure numbering still needs grouping those by staff and ordering them. YOLO’s output can be post-processed: sort barline boxes left-to-right within each staff system, then assign measure numbers incrementally. An alternative explored by

Gonzalez et al. is **direct measure detection with YOLO** – they trained a YOLOv8 model to output bounding boxes around entire measures ¹⁰. This implicitly learns to combine barlines and the intervening music into one object. They report successful sample predictions of measure segments using this approach ³⁷ ³⁸. In practice, measure detection could simplify numbering (each detection is a measure, numbered by topological order). However, it's a harder task and may require more training data with measure-level annotations. For your pipeline, it may suffice to detect barlines and derive measures from them (especially since you can identify the start of the piece and end barline easily).

Strengths: YOLO is **fast and lightweight**, suitable for real-time or batch processing on an RTX 4060. In testing, even the largest YOLOv8 model (YOLOv8x) trained on OMR achieved 99.8% precision on many symbol classes and very high recall ³⁹ ⁴⁰. It excels at regular, well-defined objects like notation symbols. It's also flexible: you can include multiple barline types as separate classes (single, double, repeat, final) and YOLO will classify each. This addresses your requirement for distinguishing repeats, final bars, etc. Another strength is that YOLO models are **publicly available and fine-tunable**. Ultralytics provides pretrained weights (trained on COCO) that one can fine-tune on musical data ¹¹. The cited project trained YOLOv8 for 1000 epochs (~8.8 hours for YOLOv8n) and improved precision/recall steadily ⁴¹, suggesting feasibility.

Weaknesses: The main challenge is training data. DeepScores is synthetic and very dense (lots of notes, high complexity) – a YOLO model solely on DeepScores may not generalize perfectly to scanned sheets with noise or slight distortions. Fine-tuning on real scanned images (even a modest number, say a few hundred annotated pages) will likely be needed to adapt the detector to your image domain. Another weakness: YOLO's trouble with **extremely thin or overlapping objects**. As noted, stems and ledger lines confused the model ⁴². Barlines could similarly be missed if they're broken or very faint. Also, YOLO by itself doesn't enforce musical consistency; it might occasionally hallucinate a barline if a tall thin artifact exists, unless the confidence threshold is carefully set. Compared to segmentation methods, a box detector might be less precise in locating the exact span of a barline (though for counting, that's minor). Finally, using the YOLO code directly introduces a GPL license issue (Ultralytics code is GPL3). You can circumvent this by re-implementing the inference with an open framework (e.g., using **Detectron2** or **MMDetection**, which are Apache-licensed, to train an equivalent model architecture on your data). The YOLO architecture itself is not patented or restricted – it's the implementation that carries GPL – so this is a solvable issue.

Applicability: A YOLO-based barline detector is a **highly promising drop-in component** for your pipeline. You could feed it the full page or region images (after staff finding). For instance, use Oemer's staff line removal as a preprocessing, then run YOLO to find vertical lines. Or provide YOLO the original image; it will find barlines along with other symbols (you can ignore the rest or even leverage them). Since YOLO can output multiple classes, you might train it to detect barlines and perhaps time signatures or clefs as context if that helps (though not strictly needed). Training on synthesized distortions of DeepScores plus a handful of real examples should get you a model that significantly reduces false positives – it will learn that a standalone vertical line is a barline, whereas a vertical line ending in a notehead is not (should be stem class). Given its speed, you can run it on each page and integrate the detected barline coordinates into your measure assembly code. In summary, object detectors like YOLO **offer a direct improvement** over heuristic filtering by learning the visual differences between barlines and similar objects, under a trainable framework.

3. Mask R-CNN Instance Segmentation

Overview: Instance segmentation extends object detection by predicting a precise pixel mask for each object, rather than just a bounding box. In OMR, this approach (exemplified by **Mask R-CNN** by He et al.)

has been used to better handle overlapping symbols ¹² ¹³ . A 2024 study by Shatri and Fazekas applied Mask R-CNN to full sheet images, comparing it to pure detection on the **DoReMi** dataset (6400 images of printed music) and MUSCIMA++ (handwritten) ¹⁴ ⁴³ . The motivation is that pixel-wise segmentation can separate intertwined symbols, such as notes on staff lines or chords, more cleanly than bounding boxes.

Barline Detection: Mask R-CNN can be **very effective for barlines**, because it not only locates them but actually outlines their shape. This means even if a barline goes through staff lines or intersects other symbols, the model can ideally produce one continuous mask for the barline. In practice, if a barline is broken (by faded ink or staff removal gaps), the segmentation model might output multiple small fragments – but instance segmentation has a chance to link them if trained on complete shapes (possibly by learning to “bridge” small gaps). The study reported substantial improvements in dense symbol environments, with instance segmentation reaching up to **59.7% mAP** (on a subset of 1384 images) as they increased training data ⁴⁴ ¹⁶ . This was comparable to their object detection baseline, but with better performance on overlapping symbols. Specifically, segmentation provided “more precise representations of musical symbols, particularly in densely populated scores” ¹⁴ . We can infer that barlines – which often overlap staff lines and sometimes touch other symbols – benefit from this pixel precision. The Mask R-CNN model can learn the shape prior that a barline is a long thin vertical mask (with possibly a slight thicker double-line shape for repeats). Stems, conversely, would be shorter vertical masks usually connected to notehead masks. Thus, by training on enough examples, the model becomes **resistant to confusion**: a predicted barline mask spanning ~5 staff spaces in height with no attached notehead mask will be labeled “barline,” whereas a vertical mask that stops at a notehead mask would likely be labeled “stem.” This instance-level understanding is closer to how a human separates symbols and is more robust than classifying individual pixels or small patches.

Measure Structure: On its own, Mask R-CNN will output each barline as an instance (with coordinates for its mask). It won't number measures automatically – you'd still need to take the detected barline positions and derive measures. That said, because segmentation is fine-grained, you get accurate positions of barline start and end on the staff. Grouping barlines into measures is similar to the YOLO case: sort by x-coordinate per staff. The segmentation doesn't directly give measure separators or groupings (unless you explicitly train a “measure” class, which they did not in the referenced study). However, one advantage is that by also detecting time signatures, clefs, etc., you could infer measure one begins after a clef/time signature at the start of a system. In general, though, **measure numbering would be a post-process**: once you have all barline masks in order, count them to number measures.

One noteworthy benefit: because Mask R-CNN can detect symbols and you can run a staff detection in parallel (as the authors did using traditional algorithms) ⁴⁵ ¹³ , you could integrate pitch/rhythm checks. For example, after detecting all symbols (notes, rests) and barlines, you can try to verify if each measure has a sensible rhythm (though that enters the realm of musical inference beyond pure vision).

Strengths: The pixel accuracy of segmentation means fewer false merges and splits. A classic problem in OMR is that touching symbols can be mistaken as one blob – Mask R-CNN mitigates that by assigning pixels to different classes. For barlines, this means things like a ledger line or staff line crossing a barline are less likely to cause the barline to vanish or fragment; the model can classify each pixel of the vertical stroke as barline even through intersections ⁴⁶ . If part of a barline is faded, the model can sometimes still “connect the dots” by shape learning. Also, Mask R-CNN retains the advantages of a trainable deep model: it can be trained on both printed and handwritten data (MUSCIMA++ has barline masks) to gain robustness to different styles. The authors emphasize that **pre-training on MUSCIMA++ (handwritten)** and then fine-tuning on DoReMi (printed) significantly improved results ¹³ – indicating the model learned general notions of notation that transfer to printed scores. In terms of tech stack, Mask R-CNN implementations are widely available (e.g., Facebook's Detectron2 is Apache-2 licensed and very well-

supported). You can leverage pretrained COCO weights for the backbone as they did, and then fine-tune on musical symbols ⁴⁷ .

Weaknesses: The main drawbacks are complexity and training cost. Mask R-CNN is heavier than YOLO – it has a two-stage detector plus a segmentation branch. Inference is slower (though on a single RTX 4060, it should still be manageable for single pages). Training requires more memory and time, especially if you have many classes. In the referenced study, even with ~1.3k images, they maxed out at ~59–60% mAP ¹⁶ , which suggests that to reach very high accuracy, you might need thousands of training images or further model tuning. Another issue: **data annotation for segmentation** is labor-intensive. You would need pixel masks for barlines (and other symbols if you train all classes). MUSCIMA++ offers this for handwritten, and DeepScores has it synthetically for printed, but DeepScores’ barline annotations had some inconsistencies (as Oemer’s authors noted). Using synthetic images for segmentation may require careful augmentation to simulate scan noise. Also, integrating Mask R-CNN’s output into your pipeline may require more adjustments – you’d get a list of masks and classes. Combining that with, say, Oemer’s output (which expects certain intermediate results) could be non-trivial. However, you could run Mask R-CNN as a replacement for Oemer’s UNet stages entirely: detect all symbols including barlines, then use your own logic to assemble the notation (HOMR’s approach of using anchors like clef/barline to organize staves is relevant here ²⁴).

Finally, from a license perspective, the reference implementation by Matterport is under MIT, and Detectron2 is under Apache, so no issues there. If you use the authors’ pretrained model (Shatri’s), you’d need to check if it’s published (they mentioned making trained models available ⁴⁸). But training your own is likely preferable to fine-tune for your specific print style and scanning artifacts.

Applicability: Mask R-CNN is a strong candidate if you aim for **precision in complex scenarios**. It shines if your scores have a lot of overlapping notation (e.g., piano grand staff with cross-staff chords, or dense orchestral scores) where object boundaries are unclear. It will likely outperform YOLO in cases of **broken or partial barlines**, because the segmentation network can still classify small fragments as part of a barline. However, this approach requires more effort to set up (collecting/annotating training data, training the model, and integrating the results). A practical path could be: start with an object detector (YOLO) for a quick win in reducing FPs; as you gather more data, consider moving to or adding Mask R-CNN for finer accuracy. One hybrid idea: use Mask R-CNN just to refine barline detection – e.g., run a faster detector to propose barline regions, then use a segmentation model to confirm and precisely locate them (this could cut down processing time). In summary, instance segmentation offers **high accuracy and resilience** at the cost of complexity, and it is a future-proof approach as OMR datasets grow.

4. DETR with Notation Graph (Transformer Detection + Assembly)

Overview: DETR (Detection Transformer) is a newer paradigm that uses an end-to-end Transformer to detect objects, removing the need for many hand-tuned components (like anchor boxes). A particularly relevant advancement is using DETR not only to detect symbols but also to predict their **relationships**, yielding a structured notation graph. A 2021 thesis by S. Allen (Aalborg Univ.) proposed a model that combines object detection and notation assembly in one step ¹⁷ ⁴⁹ . The architecture uses a CNN (ResNet-50) to extract features, then a Transformer encoder-decoder to produce a set of object outputs and an adjacency matrix of relationships between objects ⁵⁰ ⁵¹ . In essence, it can learn musical rules like “stems attach to noteheads” or “an accidental attaches to a note”. Barlines in this model are just another object class, and importantly, they typically have **no attached relationships** except possibly to a repeat-dot symbol or an ending bracket.

Barline Detection: The DETR approach is inherently **context-aware**. During training on, say, MUSCIMA++ (which has annotations linking stems to noteheads, barlines to repeat signs, etc.), the model learns that certain object pairings are valid. For instance, a vertical line object with a notehead object attached midway is likely a stem-notehead pair (so the model might classify the line as a stem and create a “notehead→stem” link). A vertical line object spanning the staff with no noteheads attached will more likely be classified as a **barline**. This relational reasoning directly addresses the confusion between stems and barlines. In other words, the model doesn’t just look at the line in isolation; it “asks”: does this line connect to something (a note)? If yes, it favors stem; if no, it favors barline. The thesis noted that a prior approach without relation learning failed to discern incompatible object combos (like linking a notehead to a barline) ⁵². Their model, by contrast, learns to avoid impossible links, effectively sharpening classification ⁵².

In terms of accuracy, while specific barline AP isn’t quoted, we have qualitative expectations: DETR should detect barlines at least as well as a CNN detector, and the added relations act as a filter for FPs. If a spuriously detected “barline” had any overlap with a notehead, the model might instead label it a stem (reducing false barlines). Conversely, a real barline that is slightly broken or faint might still be detected because the model can use the global image context (e.g., alignment across staves) to identify it. Also, DETR can naturally handle **multi-staff barlines** (like grand staff connectors) by possibly outputting one object that spans multiple staves or multiple barline objects with a relation indicating a system break. The MUSCIMA++ data actually contains a notion of “measure separator” as a higher object that groups barlines ⁵³ ⁵⁴, and indeed the thesis references a measureSeparator class (with thousands of instances) in the dataset ¹⁹. This implies the model could learn to output a measure separator token for end-of-system barlines versus regular barlines, improving structure recognition.

Measure Structure: DETR’s output can include measure-level information in a couple of ways. If trained on MUSCIMA++ fully, it could output two classes related to measures: individual barline objects and a higher-level “measure separator” object that signifies a barline at end of system (MUSCIMA++ defines measure separators as the group of barlines that align vertically at a system break) ⁵⁵. In one table, we see “barline” and “measureSeparator” listed separately ¹⁹. The model achieving good predictions for measureSeparator would essentially be identifying where measures end across systems. Even without an explicit measure class, DETR could encode measure groupings via the learned attention patterns: one could post-process the graph of barlines and staff assignments to figure out measure indices. For measure numbering, you would still count through the detected barlines in reading order. However, a unique advantage is that DETR’s output is a structured **graph** of notation, so it inherently organizes symbols by measure if correctly learned. For example, notes between barlines might all be connected in a subgraph. While the thesis work doesn’t explicitly claim measure numbering, having the notation graph would simplify adding that layer (since you know which barlines belong to which system and the sequence of notes between them).

Strengths: The transformer-based approach is **semantically rich**. It essentially embeds some musical knowledge (like “a measure is delimited by barlines”) into the model through training on realistic data. The result is fewer false linkages and likely fewer false barlines. If the model is uncertain whether a vertical line is a stem or barline, the presence of a notehead at one end would tip it toward stem because that yields a consistent graph (notehead–stem link) whereas calling it a barline would leave a notehead unattached (inconsistent). This kind of global consistency check is something other models lack. Another strength is **unified output** – one model gives you symbols and their relations, so you don’t need separate heuristics to group barlines across staves or attach repeat dots. The DETR will, for instance, learn to attach repeat dots to a barline object (so you know immediately if a barline is a repeat barline with dots, which is useful for notation). License-wise, DETR code (from Facebook) is Apache, and the concept is well-known, so implementation can be done in PyTorch without issues. It’s also worth noting that DETR

doesn't require non-maximum suppression or anchor tuning; it outputs a fixed number of object slots. This can simplify training (no need to handle extreme class imbalance in proposals as much).

Weaknesses: This is bleeding-edge for OMR. The approach has been proven in research prototypes but not widely deployed yet. Training DETR for OMR requires **extensive annotated data with relations**. MUSCIMA++ (140 images) is great for handwritten but small; DeepScores provides no semantic relations (just symbols). One might combine symbol detection data (for visual training) with a smaller set of fully annotated images (for relations). The model might also have trouble if the training data doesn't cover certain notational conventions – for instance, if measure separators (end-system barlines) aren't well represented, it might not handle multi-system numbering perfectly. DETR is also computationally heavy: the self-attention on high-resolution feature maps can use a lot of memory. Typically, DETR models operate on images up to 800–1000px; full sheet images (2500px) might need tiling or a reduced scale. The Aalborg thesis likely dealt with single-staff images or had to downscale. Running a DETR on a full page at 300 DPI could be slow or require a smaller scale (which risks missing very thin lines). There's also the issue of **post-processing**: while DETR gives relations, your system would need to interpret them (which is extra coding). For example, you'd get an adjacency matrix of size $[N_objects \times N_objects]$ where an entry might indicate "object i is attached to object j." You'd need to parse out that barlines have no parent except maybe a system object, etc. This is doable with some logic.

Applicability: If you're looking to invest in a next-generation solution, a DETR-based model is a compelling direction. In practical terms, adopting it now would mean significant R&D: you'd have to assemble training data (possibly render synthetic scores with known ground truth linking each note to a measure index, etc., or leverage MUSCIMA++ and augment it). The reward would be a single model that could replace large parts of the pipeline – not just barline detection, but symbol classification and even some musical understanding. In the context of measure numbering, DETR could make life easier by explicitly telling you where the measures end (since it can identify the "end of line" barlines). It might also help detect missing barlines (e.g., if a measure has too many beats and no barline, the model might insert one or flag it by the rhythm relation stage – though that's speculative). For now, **DETR is promising but experimental**. It could be something to watch or gradually experiment with (perhaps start by training DETR just for object detection of symbols to see performance, then add relation learning). Given that your immediate issue is false positives that heuristics can't solve, DETR's global reasoning is a future fix. It's probably an overkill for a quick improvement, but it points the way to an OMR system that inherently avoids many logical errors.

In summary, DETR with notation assembly would **significantly reduce stem-vs-barline confusion** and align with measure structure out-of-the-box, but expect a heavier lift in implementation and training. If successful, it would yield very clean measure numbering because the model effectively "understands" the concept of measures through learning the layout and sequence of symbols.

5. End-to-End Sequence Models (Full-Page OMR)

Overview: End-to-end OMR treats the problem more like a translation: input an image, output a sequence of music symbols (in a format like MusicXML, MEI, or a custom encoding). Traditionally, this was done on **single staff images** using CNN+LSTM with CTC or attention (similar to how OCR works for text lines) ⁵⁶ ⁵⁷. Recently, **Transformer architectures** have entered this space. Calvo-Zaragoza et al.'s **Sheet Music Transformer (SMT)** was one of the first to handle full pages with a Transformer-based seq2seq model ²⁰. They demonstrated it on piano scores, outputting a sequence that represents the music. More recently, an **implicit layout-aware Transformer (LA-SMT)** extends this to also infer layout (like staff regions) from the attention maps ²² ²¹. In these models, barlines appear as tokens in the output sequence (for example, a  symbol or an XML `<measure>` tag) indicating measure breaks.

Barline Detection: Unlike detectors, a seq2seq model doesn't explicitly "detect" barlines in the image. Instead, it **learns to insert barline tokens at appropriate points in the sequence output**. This decision is based on both visual cues and learned musical patterns. For instance, the model might learn that a vertical line spanning the staff corresponds to a barline token. Or, if a measure's worth of notes has passed (based on time signature context), it expects a barline in the sequence. The accuracy of barline placement by such models can be very high in terms of musical correctness: they tend not to output extra barlines because that would violate learned syntax (e.g., producing an empty measure with no notes is unlikely unless there is a rest). In fact, one reason to favor end-to-end is that it inherently avoids certain nonsensical outputs – it's **constrained by the training data's musical syntax**. As an example, Audiveris (classical pipeline) often could find barlines but might group measures incorrectly; a learned sequence model would "know" typical measure patterns and thus might be more **false-positive resistant**. If there's a stray mark that looks like a barline but contextually doesn't fit (no corresponding measure in the music), the model might ignore it. Indeed, an MIT study pointed out that focusing on barlines (rather than inferring measures) is easier, and measure interpretation should be handled at a higher level ⁵⁸. Sequence models essentially handle that higher-level interpretation internally.

However, it's worth noting that early end-to-end models (like those using CTC) occasionally struggled with **split or missing barlines** if the visual input was ambiguous – they might concatenate measures or drop a barline token if the image was unclear, leading to rhythmic errors. Newer Transformer-based models with attention might do better by "seeing" the entire page context. The LA-SMT paper specifically aimed to retrieve structural information (like staff divisions and possibly measure positions) from the model's attention, which suggests that the model's attention heads often focus on barline regions when outputting a barline token ⁵⁹ ⁶⁰. So effectively, the model is detecting barlines implicitly to decide when to output . Overall, when properly trained, end-to-end OMR has high precision on barline placement because it must produce a valid musical sequence. False barlines (extras) would make the sequence invalid under the learned music grammar (unless the model grossly errs).

Measure Numbering: Measure numbering becomes trivial – the model's output is an ordered sequence of measures. If outputting MusicXML, measures are numbered sequentially by definition, or you can count the  tokens. End-to-end systems naturally handle picks-ups and partial measures if trained on such data: for example, they might start with an incomplete measure without a leading barline token (common practice). The output will be a continuous stream, so the concept of measure numbers is just counting in order. One caveat: if the model mis-segments (e.g., merges two measures into one because it missed a barline), all subsequent measure numbers would be off by one. But this is easy to detect by comparing to an expected number of measures or checking time signature consistency (if available).

A big advantage is **rhythmic validation**: many sequence models either explicitly or implicitly enforce that the notes in each measure sum up correctly to the time signature. For instance, some use a rhythm encoding or a grammar that doesn't allow too many beats in a measure. The Transformer can learn these constraints. Thus, measure boundaries that would cause rhythm errors are less likely to appear in the output. This implicitly helps measure numbering accuracy – you won't get an extra barline that creates a 0-duration measure or splits a half note incorrectly, etc.

Strengths: The end-to-end approach is the holistic solution. It simultaneously addresses symbol recognition, relation (the order of notes and measures), and output formatting. For measure-related tasks, it means **no separate barline detection stage is needed** – the model figures it out. If your current pipeline is struggling to reduce FPs without losing info, an end-to-end model might bypass that struggle entirely by working in a completely data-driven fashion. Users have reported that modern end-to-end systems (like Calvo-Zaragoza's) produce very pleasing results on clean images ⁶¹. Also, these models can be trained to be **agnostic to noise** by data augmentation – e.g., random blotches or distortions – and

since they consider global context, a local artifact is less likely to throw off the output if the rest of the measure is clear.

From a maintenance perspective, having one model might simplify the pipeline (no hand-tuning of thresholds for barlines, etc.). And many sequence models are open-source in research. For example, there's an implementation by Ríos-Vila et al. exploring Transformers for OMR ⁶² ⁶³. They concluded vanilla Transformers didn't yet beat LSTMs in 2022, but the field has advanced. The LA-SMT (2025) improved by adding layout extraction, showing this is an active area with improving performance.

Weaknesses: Training an end-to-end model is **data-hungry**. You need a large corpus of sheet images with corresponding notation ground truth. Synthetic data can be used (e.g., rendering MusicXML to images), but you must cover various layouts, fonts, and ensure barline appearances match scanned ones. Also, these models can be heavy: full-page Transformers can have millions of parameters and may need GPU memory optimization. Inference time might be slower than a dedicated detector because the model is essentially “reading” the entire score symbol by symbol. For instance, OMR sequence models can take a few seconds per page on GPU, depending on complexity.

Another risk: **lack of interpretability and fine control**. If the model does make a mistake (say it skips a barline it should have output), it's not straightforward to fix without retraining or adding a corrective heuristic. By contrast, with a modular pipeline, you could tweak the barline detector threshold if you notice a consistent issue. End-to-end is a black box to some extent. The training process also must balance all aspects of recognition – it might trade off a little barline accuracy to improve note accuracy, depending on the loss function. This can be mitigated by using a structured output format and perhaps multi-task learning (predict sequence + predict some intermediate like staff line positions).

Finally, license and integration: Most end-to-end OMR code is research under permissive licenses, but not necessarily plug-and-play. You might have to adapt an academic implementation to your use case. Also, adopting an end-to-end model means **replacing** much of your existing pipeline (staff detection, symbol classification, etc.), which is a big shift. It might be overkill if your only major problem is barline FPs. However, if your ultimate goal is to improve the entire OMR accuracy (not just barlines), then this route could yield the highest long-term gains.

Applicability: For your specific need – accurate barlines and measure numbering – an end-to-end model will definitely achieve that **when it works**. In fact, measure numbering becomes a non-issue; you'll get measures inherently. But the question is whether it's practical to deploy. If you have a moderate number of pages to process and can tolerate some setup time, you could try fine-tuning an existing model (like Calvo's SMT) on a small set of your sheets to see how it handles them. The model could also be used as a “second opinion”: run your pipeline and the end-to-end model, cross-check the measures count and positions. A discrepancy (like the end-to-end suggests a barline where your pipeline didn't detect one) could signal a missed or false barline.

Given your current pipeline's foundation in Oemer/HOMR, a multi-stage system, shifting to end-to-end is a paradigm change. It **simplifies measure handling** tremendously at the cost of requiring a robust model. This approach is promising especially if you foresee expanding to more varied music or if maintaining many heuristic fixes is cumbersome. It's also the direction many OMR researchers are heading, so investing in it could future-proof your solution. Just keep in mind that in the short term, training such a model to surpass your current system's accuracy might be challenging without a lot of training data. A reasonable strategy could be to start leveraging end-to-end as a helper (for example, use it to generate synthetic annotations or to validate measure counts) while improving your pipeline with the other techniques.

Recommendations and Next Steps

Considering the above, here's a **multi-pronged recommendation** tailored to your use case (commercial-friendly, 200–300 DPI scans of printed music, focus on barlines):

1. Integrate a Dedicated Barline Detector (High Priority): The quickest win is to add a modern detection module for barlines to reduce false positives. Training a YOLOv5/8 or Mask R-CNN model on synthetic and a bit of real data is very feasible on an RTX 4060. For a start, try YOLO: it's easier to set up and very fast. You can use DeepScores (printed) as a base – it has labeled barlines (including repeat and double barlines). Fine-tune this with ~100–200 real scanned pages (grayscale, 300 DPI) that you annotate for barlines. Annotation isn't too painful: you can use something like Labellmg for boxes or CVAT for polygons (if doing masks). This model, once trained, would sit in your pipeline after staff detection: it takes an image (or a staff-cropped image) and outputs barline locations. Use a high confidence threshold (to be conservative) – any detected barline is almost certainly real ³⁹ ⁴⁰. Early experiments suggest you can get precision >95% easily, meaning very few FPs, and recall high enough to catch all real barlines. This alone addresses your main problem: you'll drop those last pesky FPs that the heuristic couldn't distinguish (because a learned model will use subtle cues to differentiate them).

2. Combine with HOMR/Oemer Outputs (Hybrid Pipeline): You don't have to throw away your existing system – instead, **use it in combination** with the new detector. For example, run Oemer to get an initial result, then run the barline detector. You can then cross-validate: only keep barlines that both approaches agree on, or use the detector's output as the truth and override Oemer's barlines. Given Oemer tends to overshoot (FPs) rather than miss barlines, a logical rule is: **accept only barlines that the learned detector finds**, and discard any extra ones Oemer had. This will immediately wipe out the false ones (which Oemer found but the detector did not). The risk is if the detector has false negatives – but with proper training, it should catch nearly all, and you can design a fallback that if Oemer found a barline the detector missed, you flag it for manual review or apply a secondary check (like is there a time signature change that must coincide with a double barline?). In our view, a trained model will recall barlines as well as Oemer, so this hybrid should strictly improve precision without hurting recall.

Additionally, HOMR's method of using clefs and barlines as anchors ²⁴ can be augmented: use the detector's barlines as anchors for splitting systems and measures, while still using Oemer/HOMR for recognizing notes and other symbols. This way, each module does what it's best at – the deep detector does structure (staff lines via Oemer's first UNet, and barlines via new model), and Oemer's second stage or an SVM handles fine symbol classification. This **modular improvement** minimizes disruption to your pipeline but yields better measure numbering (since measure boundaries will be correct).

3. Utilize Data from Public Datasets (MUSCIMA++, DeepScores, DoReMi): To train and evaluate, leverage what's available: - **DeepScores** (for printed) or its newer version DeepScoresV2: Use this for initial training of symbol detectors. It's huge, so you can even filter it to just classes you care about (barlines, maybe clefs and basic symbols to give context). DeepScores will provide examples of all barline types (including repeats with dots, final double bars). - **MUSCIMA++** (for handwritten, but useful): It has only 140 images, but each has pixel masks for barlines and a graph of relationships. You can use it to test a Mask R-CNN or DETR approach on a smaller scale. If you train a Mask R-CNN on MUSCIMA++ barlines, you'll get a feel for performance (the model should learn to segment barlines even when broken by removal – if it works on those, it will work on printed). - **DoReMi** (newer dataset of 6400 printed images): This would be a goldmine for training, but it may not be publicly downloadable yet (it was introduced in 2021 ⁶⁴). If it becomes available (perhaps through contacting the authors or if they release it), it would give you plenty of real printed training data for segmentation/detection, including barlines. - Also, consider **Audiveris outputs** as a pseudo-dataset: since Audiveris is quite accurate on barlines ⁵⁸, you

could run Audiveris on a collection of public-domain scores and gather its barline detections (especially if you use the JSON output modification ⁶⁵). Those can serve as additional training annotations or at least validation for your model (bearing in mind Audiveris might miss some or add some false ones with broken lines ⁶⁶).

By combining these data sources, you minimize manual labeling. You might only need to manually annotate a small set of your specific scores to fine-tune.

4. Evaluate Transformer-Based Methods (Long Term): For immediate needs, end-to-end might be excessive, but keep an eye on it. As a pilot, you could test a pre-trained model (if any exist, e.g., Calvo's group might have a demo for their sequence model on Printed scores). See how it handles one of your pages – does it correctly number measures? If yes, it might be a benchmark to aspire to. If not, note the errors (are barlines missed or extra?). This can guide whether investing in training your own end-to-end model is worthwhile. In a few years, end-to-end OMR might become the norm (the LA-SMT paper shows the trend ²⁰ ²¹), so being familiar with it is good. But for now, a practical approach is to **use end-to-end as validation**: for example, after your pipeline, run a quick sequence model that outputs a MusicXML and check if the number of measures or barline positions differ. Discrepancies could then be resolved by either model or flagged.

5. License Considerations: All suggested approaches can be done in a commercial-friendly way: - Use Oemer (MIT) as base. HOMR's improvements (AGPL) you can re-implement in your code if needed (e.g., the idea of using a Transformer for sequence decoding – you don't have to use HOMR's code, you can write your own or use PyTorch implementations which are MIT). - YOLO: Avoid Ultralytics' code in embedded form. Instead, use an open implementation or treat it as an external service. For example, train with Ultralytics (since training doesn't infect your product), then export the weights and write a small inference script with ONNX or LibTorch. The model weights are your own and not GPL. This way you get YOLOv8 performance without license issues. - Mask R-CNN/Detectron2: completely fine (Apache License). - DETR: also fine (Apache). If you adapt the thesis code, just check if they provided any license – otherwise implement using Facebook's DETR repo (MIT license). - End-to-end: most academic code is under MIT or similar. Just double-check, but writing from scratch with PyTorch/TensorFlow is always an option if needed.

6. Additional Dataset Needs: In summary, you should **collect a small custom dataset** of your target domain: a mix of PDF-exported images and actual scans if available. Annotate barlines (and ideally repeats/double separately). Even 50 pages could be enough to tune a model that was pretrained on synthetic data. Augment this with synthetic distortions (blur, speckle, staff line thickness variation, slight rotation) so the model sees the kind of “artifacts” you mentioned (fading, noise). If measure numbering should handle pickup measures or absent initial barline, include some examples of those in training (so the model doesn't always expect a barline at far left). Also, since you might care later about other symbols (rehearsal marks, segno, etc.), keep your dataset extensible – but for now focus on barlines and maybe basic clef/time, as those help structure.

7. Testing and Validation: Once you have a new barline detection mechanism, test it on a set of challenging samples: cases with many short measures (lots of barlines), pieces with dotted barlines or sectional double bars, and pieces with thick barlines (final). Check that repeats  are detected as two barline objects or a special class. Ensure that the output measure count matches the expected count from the score. It may be useful to use MusicXMLs from reliable sources (or MIDI exports) to validate measure numbering. For example, if you have the ground truth MusicXML for a page, compare the barline count and positions.

8. Continuous Improvement: With your improved pipeline (staff detection from Oemer, barlines from new model, note symbols from Oemer/HOMR or even Audiveris as cross-check), you can likely achieve a much lower false positive rate for barlines. This means **measure numbering will be highly reliable**. The few remaining issues might be: occasionally a very faint barline gets missed – you could catch this if the sum of note durations in a measure far exceeds the time signature (then you know a barline was missed and can search for a faint line or prompt manual review). Or a very thick noise line gets falsely picked up – but a well-trained model should be robust to random vertical noise if it's not exactly between notes like a barline would be.

In conclusion, the **recommended approach** is a **multi-stage hybrid**: keep the best of your current pipeline (which already segments and identifies symbols well) and inject a state-of-the-art detection module for barlines. Among the models, a **YOLO/Mask R-CNN** solution is likely to give immediate benefits. Looking forward, **transformer-based models** (DETR for detection or full seq2seq) offer exciting improvements by reducing logic errors and could be gradually introduced as your project grows. This strategy will yield **accurate barline detection and robust measure numbering**, meeting the core objective with a balance of short-term impact and long-term innovation.

References:

- Y. Su et al., “Oemer: End-to-end OMR with Deep Learning.” GitHub (2023). [Technical details and two-UNet architecture] [1](#) [3](#)
- C. Liebhardt, “HOMR: Deep Learning OMR Pipeline.” GitHub (2023). [Builds on Oemer, uses segmentation to find clefs & bar lines as anchors for staff assembly] [2](#) [24](#)
- D. Gonzalez et al., OMR with YOLO and Faster R-CNN. GitHub (2023). [Trained YOLOv8 on DeepScoresV2 for symbols and measures; achieved mAP $\approx$ 0.728, with stem detection remaining challenging] [35](#) [8](#)
- A. Pacha, J. Hajič jr., et al., “Baseline for General Music Object Detection,” Appl. Sci. 8(9):1488, 2018. [Compared Faster R-CNN, RetinaNet, U-Net on MUSCIMA and DeepScores; notes that overlapping symbols (stems vs barlines) require special handling] [67](#) [33](#)
- E. Shatri, G. Fazekas, “Enhancing OMR with Instance Segmentation,” arXiv:2408.15002 (2024). [Mask R-CNN on DoReMi & MUSCIMA++; instance segmentation improves symbol detection in dense scores, reaching 59.7% mAP] [14](#) [15](#)
- S. Allen, “Combined Object Detection and Notation Assembly in OMR,” MSc Thesis, Aalborg Univ. (2021). [Introduces DETR for OMR; outputs notation graph with relationships (notehead–stem, etc.), improving semantic consistency] [17](#) [51](#)
- A. Ríos-Vila et al., “On the Use of Transformers for End-to-End OMR,” IbPRIA 2022. [Explores transformer vs CRNN for sequence transcription; highlights challenges and future direction for full-page models] [62](#) [63](#)
- J. Calvo-Zaragoza et al., “Implicit Layout-Aware Sheet Transformer,” Int. J. Multimedia Info. Retr. (2025). [Full-page Transformer OMR that also outputs layout; first to integrate transcription with structural analysis in one step] [20](#) [21](#)

- H. Bitteur, “Audiveris 5.3 – Open-Source OMR,” Audiveris Handbook (2025). [Classical OMR engine; uses ad-hoc line detection for barlines and achieves high barline accuracy on well-scanned music]

58 68

- T. Wilmott, “Graphical Feedback for Conductors,” MIT (2017). [Used Audiveris to extract barlines for a conductor feedback system; notes Audiveris barline detection is more reliable than measure reconstruction]

58 65

1 3 4 5 23 25 27 28 29 31 61 GitHub - BreezeWhite/oemer: End-to-end Optical Music Recognition (OMR) system. Transcribe phone-taken music sheet image into MusicXML, which can be edited and converted to MIDI.

<https://github.com/BreezeWhite/oemer>

2 24 30 GitHub - liebharz/homr: homr is an Optical Music Recognition (OMR) software designed to transform camera pictures of sheet music into machine-readable MusicXML format.

<https://github.com/liebharz/homr>

6 7 8 9 10 11 35 36 37 38 39 40 41 42 GitHub - dmgonzalez8/OMR

<https://github.com/dmgonzalez8/OMR>

12 13 14 15 16 43 44 45 47 48 64 Knowledge Discovery in Optical Music Recognition: Enhancing Information Retrieval with Instance Segmentation

<https://arxiv.org/html/2408.15002v1>

17 18 19 49 50 51 52 projekter.aau.dk

https://projekter.aau.dk/projekter/files/414945223/mi1014f21_thesis.pdf

20 21 22 56 57 59 60 An implicit layout-aware transformer for full-page end-to-end optical music recognition | International Journal of Multimedia Information Retrieval

<https://link.springer.com/article/10.1007/s13735-025-00385-5>

26 58 65 [PDF] Automating Graphical Feedback for ConcertCue.com's Human ...

https://musictech.mit.edu/wp-content/uploads/2017/07/wilmot_uap.pdf

32 33 34 67 A Baseline for General Music Object Detection with Deep Learning | MDPI

<https://www.mdpi.com/2076-3417/8/9/1488>

46 66 68 Audiveris Handbook | PDF | Java (Programming Language) | Microsoft Windows

<https://www.scribd.com/document/868091441/Audiveris-Handbook>

53 54 55 GitHub - OMR-Research/muscima-pp: Working repository for the MUSCIMA++ dataset

<https://github.com/OMR-Research/muscima-pp>

62 63 (PDF) On the Use of Transformers for End-to-End Optical Music Recognition

https://www.researchgate.net/publication/360183761_On_the_Use_of_Transformers_for_End-to-End_Optical_Music_Recognition